

Agent Skill — eshop-automation

Sinh viên	23127195
Skill	eshop-automation
Vị trí trong repo	<code>.claude/skills/eshop-automation/SKILL.md</code>
Video demo	https://youtu.be/Vxf9-R9AC54
Feature dùng để demo	FR-05 — Tìm kiếm sản phẩm (ngoài ba feature nộp)

1. Skill này làm gì

Trong lúc làm ba feature FR-01, FR-09, FR-14 tôi mất khá nhiều lần sửa đi sửa lại cho cùng một loại lỗi: selector đoán sai vì DOM của SUT không chuẩn, assertion báo xanh giả vì chạy trước khi SPA kịp phản hồi, test đầu độc dữ liệu của nhau, hay một bản vá lại làm hỏng test khác. `eshop-automation` gói lại những bài học đó thành một quy trình 8 bước để lần sau không giẫm lại vết cũ:

1. Đọc đặc tả trước khi đọc mã nguồn — lấy đặc tả làm chuẩn cho kỳ vọng, không lấy hành vi hiện có.
2. Probe DOM và hành vi thật bằng một script vớt đi, rồi mới viết selector.
3. Để mọi giá trị đầu vào ra file `tests/data/` (CSV cho ma trận, JSON cho case).
4. Viết assertion không race — tuyệt đối không chứng minh "không xảy ra" bằng việc URL chưa đổi.
5. Mỗi test tự cô lập và tự dọn ở cả hai nhánh pass/fail.
6. Chạy từng engine, mỗi engine một report.
7. Kiểm chứng bằng cách đọc *lý do* fail, không đếm số.
8. Ghi nhận bug: bug report, ảnh, GitHub issue.

Skill còn kèm một bảng "SUT facts" ghi sẵn những cạm bẫy đã tốn công phát hiện (ví dụ `getByLabel` vô dụng ở SUT này, mật khẩu admin thật là `Admin123!` chứ không phải `admin123`, cổng của từng service, cách reseed...).

2. Buổi demo trên FR-05

Video 2 quay lại cảnh áp skill lên một feature chưa từng đụng tới: ô tìm kiếm sản phẩm ở trang chủ. Làm đúng theo skill, bước probe đọc DOM thật và thử vài payload trước khi viết bất kỳ assertion nào. Kết quả bộ test `tests/fr05-search.spec.js` gồm 21 test case, chạy trên cả 3 engine cho ra **16 pass / 5 fail** giống hệt nhau, và cả 5 test đỏ đều gắn `@bug`.

Năm test đỏ đó là hai lỗi hồng bảo mật thật, không phải test viết sai:

Test	Lỗi hồng	Bằng chứng
TC-12, TC-13	Reflected XSS. Ô tìm kiếm nhét thẳng chuỗi người dùng vào DOM qua <code>dangerouslySetInnerHTML</code> . Payload <code></code> chạy được mã JS.	<code>evidence/BUG-16-search-reflected-xss.png</code>
TC-14	SQL injection. GET <code>/api/products?search=</code> nối chuỗi trực tiếp vào câu <code>LIKE</code> . Từ khoá <code>' OR '1'='1</code> trả về toàn bộ sản phẩm dù chẳng sản phẩm nào tên như vậy.	<code>evidence/BUG-17-search-sql-injection.png</code>
TC-15, TC-16	Ký tự <code>%</code> hoặc <code>'</code> làm vỡ câu truy vấn, server trả 500 và rò rỉ nguyên câu lỗi SQL ra client.	<code>evidence/BUG-18-search-sql-error-leak.png</code>

Một chi tiết đáng nhớ từ buổi demo (ghi lại thành R-13 trong báo cáo chính): lúc đầu định thử payload `<svg onload=...>`, nhưng probe cho thấy nó không chạy khi chèn qua `innerHTML`, nên nếu tin vào đó thì test sẽ xanh giả và bỏ sót cả lỗi hồng. Phải đổi sang `<img src=x onerror=...>` mới lộ ra XSS. Đây đúng là kiểu bẫy mà bước probe của skill sinh ra để tránh.

3. Vì sao FR-05 không nằm trong bài nộp tính điểm

FR-05 chỉ đóng vai trò minh hoạ cho Agent Skill. Ba feature được chấm vẫn là FR-01, FR-09, FR-14 như đã chọn từ HW02. Toàn bộ test, dữ liệu và report của FR-05 nằm trong repo (phần Agent Skill), còn ba bug XSS/SQLi ở trên được để riêng tại đây, không trộn vào 15 bug của ba feature chính.